Lecture 4: Real-Coded Genetic Algorithm (RGA)

Course: Evolutionary Algorithms

Instructor: Engr. Muhammad Farooq

Learning outcomes

By the end of this lecture, students should be able to:

- explain why binary coding becomes inconvenient for continuous variables

Learning outcomes

By the end of this lecture, students should be able to:

- explain why binary coding becomes inconvenient for continuous variables
- describe the overall working flow of a real-coded genetic algorithm

Learning outcomes

By the end of this lecture, students should be able to:

- explain why binary coding becomes inconvenient for continuous variables
- describe the overall working flow of a real-coded genetic algorithm
- evaluate a small real-valued population for a minimization problem

Learning outcomes

By the end of this lecture, students should be able to:

- explain why binary coding becomes inconvenient for continuous variables
- describe the overall working flow of a real-coded genetic algorithm
- evaluate a small real-valued population for a minimization problem
- perform binary tournament selection step by step

Learning outcomes

By the end of this lecture, students should be able to:

- explain why binary coding becomes inconvenient for continuous variables
- describe the overall working flow of a real-coded genetic algorithm
- evaluate a small real-valued population for a minimization problem
- perform binary tournament selection step by step
- explain the idea of simulated binary crossover (SBX)

Learning outcomes

By the end of this lecture, students should be able to:

- explain why binary coding becomes inconvenient for continuous variables
- describe the overall working flow of a real-coded genetic algorithm
- evaluate a small real-valued population for a minimization problem
- perform binary tournament selection step by step
- explain the idea of simulated binary crossover (SBX)
- explain polynomial mutation for real numbers

Learning outcomes

By the end of this lecture, students should be able to:

- explain why binary coding becomes inconvenient for continuous variables
- describe the overall working flow of a real-coded genetic algorithm
- evaluate a small real-valued population for a minimization problem
- perform binary tournament selection step by step
- explain the idea of simulated binary crossover (SBX)
- explain polynomial mutation for real numbers
- perform one full generation of RGA by hand on a small example

Learning outcomes

By the end of this lecture, students should be able to:

- explain why binary coding becomes inconvenient for continuous variables
- describe the overall working flow of a real-coded genetic algorithm
- evaluate a small real-valued population for a minimization problem
- perform binary tournament selection step by step
- explain the idea of simulated binary crossover (SBX)
- explain polynomial mutation for real numbers
- perform one full generation of RGA by hand on a small example
- explain survivor selection using the $(\mu + \lambda)$ strategy

Connection with previous lectures

In Lecture 2 and Lecture 3, we studied:

- binary representation of chromosomes

Connection with previous lectures

In Lecture 2 and Lecture 3, we studied:

- binary representation of chromosomes
- binary tournament selection

Connection with previous lectures

In Lecture 2 and Lecture 3, we studied:

- binary representation of chromosomes
- binary tournament selection
- crossover and mutation in binary-coded GA

Connection with previous lectures

In Lecture 2 and Lecture 3, we studied:

- binary representation of chromosomes
- binary tournament selection
- crossover and mutation in binary-coded GA
- creation of the next generation

Connection with previous lectures

In Lecture 2 and Lecture 3, we studied:

- binary representation of chromosomes
- binary tournament selection
- crossover and mutation in binary-coded GA
- creation of the next generation

Today's question: What should we do when the decision variables are naturally real numbers, such as lengths, voltages, gains, temperatures, or coordinates?

Why binary coding can be problematic in continuous search

Issue 1: Binary GA makes the search space discrete

A continuous variable is forced into a finite number of binary levels.

- The search is no longer truly continuous.

Why binary coding can be problematic in continuous search

Issue 1: Binary GA makes the search space discrete

A continuous variable is forced into a finite number of binary levels.

- The search is no longer truly continuous.
- The precision depends on the number of bits used.

Why binary coding can be problematic in continuous search

Issue 1: Binary GA makes the search space discrete

A continuous variable is forced into a finite number of binary levels.

- The search is no longer truly continuous.
- The precision depends on the number of bits used.
- Higher precision requires longer strings.

Why binary coding can be problematic in continuous search

Issue 1: Binary GA makes the search space discrete

A continuous variable is forced into a finite number of binary levels.

- The search is no longer truly continuous.
- The precision depends on the number of bits used.
- Higher precision requires longer strings.
- Longer strings increase representation overhead.

Why binary coding can be problematic in continuous search

Issue 1: Binary GA makes the search space discrete

A continuous variable is forced into a finite number of binary levels.

- The search is no longer truly continuous.
- The precision depends on the number of bits used.
- Higher precision requires longer strings.
- Longer strings increase representation overhead.

For many engineering problems, the original variables are already real valued. Encoding them into bits is often unnecessary extra work.

Two classic difficulties in binary coding

1. Hamming cliff

- Binary strings 10000 and 01111 decode to 16 and 15.

Two classic difficulties in binary coding

1. Hamming cliff

- Binary strings 10000 and 01111 decode to 16 and 15.
- Their numerical values are adjacent.

Two classic difficulties in binary coding

1. Hamming cliff

- Binary strings 10000 and 01111 decode to 16 and 15.
- Their numerical values are adjacent.
- But all bits are different.

Two classic difficulties in binary coding

1. Hamming cliff

- Binary strings 10000 and 01111 decode to 16 and 15.
- Their numerical values are adjacent.
- But all bits are different.

A very small move in decoded numerical space may need a very large change in the binary string.

Two classic difficulties in binary coding

1. Hamming cliff

- Binary strings 10000 and 01111 decode to 16 and 15.
- Their numerical values are adjacent.
- But all bits are different.

A very small move in decoded numerical space may need a very large change in the binary string.

2. Fixed precision

Two classic difficulties in binary coding

1. Hamming cliff

- Binary strings 10000 and 01111 decode to 16 and 15.
- Their numerical values are adjacent.
- But all bits are different.

A very small move in decoded numerical space may need a very large change in the binary string.

2. Fixed precision

$$\Delta x = \frac{x_i^{(U)} - x_i^{(L)}}{2^l - 1}$$

Two classic difficulties in binary coding

1. Hamming cliff

- Binary strings 10000 and 01111 decode to 16 and 15.
- Their numerical values are adjacent.
- But all bits are different.

A very small move in decoded numerical space may need a very large change in the binary string.

2. Fixed precision

$$\Delta x = \frac{x_i^{(U)} - x_i^{(L)}}{2^l - 1}$$

- Here l is the number of bits used for variable x_i.
- With fixed l, the precision is fixed.
- To get finer precision, we must increase the string length.

Remedy: use real numbers directly

- In a real-coded genetic algorithm, each chromosome is a vector of real numbers.

Remedy: use real numbers directly

- In a real-coded genetic algorithm, each chromosome is a vector of real numbers.
- No binary encoding is required.

Remedy: use real numbers directly

- In a real-coded genetic algorithm, each chromosome is a vector of real numbers.
- No binary encoding is required.
- No decoding is required.

Remedy: use real numbers directly

- In a real-coded genetic algorithm, each chromosome is a vector of real numbers.
- No binary encoding is required.
- No decoding is required.
- The representation matches the original optimization problem more naturally.

Remedy: use real numbers directly

- In a real-coded genetic algorithm, each chromosome is a vector of real numbers.
- No binary encoding is required.
- No decoding is required.
- The representation matches the original optimization problem more naturally.

Important point: Selection remains conceptually the same because it only needs fitness values. The main changes happen in crossover and mutation.

Generalized framework of evolutionary computation

Generic algorithm

1. choose solution representation

Generalized framework of evolutionary computation

Generic algorithm

1. choose solution representation
2. set generation counter $t = 1$ and maximum generation T

Generalized framework of evolutionary computation

Generic algorithm

1. choose solution representation
2. set generation counter $t = 1$ and maximum generation T
3. generate an initial parent population $P(t)$

Generalized framework of evolutionary computation

Generic algorithm

1. choose solution representation
2. set generation counter $t = 1$ and maximum generation T
3. generate an initial parent population $P(t)$
4. evaluate $P(t)$ and assign fitness

Generalized framework of evolutionary computation

Generic algorithm

1. choose solution representation
2. set generation counter $t = 1$ and maximum generation T
3. generate an initial parent population $P(t)$
4. evaluate $P(t)$ and assign fitness
5. while $t \leq T$ do

Generalized framework of evolutionary computation

Generic algorithm

1. choose solution representation
2. set generation counter $t = 1$ and maximum generation T
3. generate an initial parent population $P(t)$
4. evaluate $P(t)$ and assign fitness
5. while $t \leq T$ do
   - $M(t) = \text{Selection}(P(t))$

Generalized framework of evolutionary computation

Generic algorithm

1. choose solution representation
2. set generation counter $t = 1$ and maximum generation T
3. generate an initial parent population $P(t)$
4. evaluate $P(t)$ and assign fitness
5. while $t \leq T$ do
   - $M(t) = \text{Selection}(P(t))$
   - $Q(t) = \text{Variation}(M(t))$

Generalized framework of evolutionary computation

Generic algorithm

1. choose solution representation
2. set generation counter $t = 1$ and maximum generation T
3. generate an initial parent population $P(t)$
4. evaluate $P(t)$ and assign fitness
5. while $t \leq T$ do
   - $M(t) = \text{Selection}(P(t))$
   - $Q(t) = \text{Variation}(M(t))$
   - evaluate $Q(t)$

In RGA, the framework stays familiar. The major redesign is in the **representation** and the **variation operators**.

Generalized framework of evolutionary computation

Generic algorithm

1. choose solution representation
2. set generation counter $t = 1$ and maximum generation T
3. generate an initial parent population $P(t)$
4. evaluate $P(t)$ and assign fitness
5. while $t \leq T$ do
   - $M(t) = \text{Selection}(P(t))$
   - $Q(t) = \text{Variation}(M(t))$
   - evaluate $Q(t)$
   - $P(t+1) = \text{Survivor}(P(t), Q(t))$

In RGA, the framework stays familiar. The major redesign is in the **representation** and the **variation operators**.

Generalized framework of evolutionary computation

Generic algorithm

1. choose solution representation
2. set generation counter $t = 1$ and maximum generation T
3. generate an initial parent population $P(t)$
4. evaluate $P(t)$ and assign fitness
5. while $t \leq T$ do
   - $M(t) = \text{Selection}(P(t))$
   - $Q(t) = \text{Variation}(M(t))$
   - evaluate $Q(t)$
   - $P(t+1) = \text{Survivor}(P(t), Q(t))$
   - $t = t + 1$

In RGA, the framework stays familiar. The major redesign is in the **representation** and the **variation operators**.

Generalized framework of evolutionary computation

Generic algorithm

1. choose solution representation
2. set generation counter $t = 1$ and maximum generation T
3. generate an initial parent population $P(t)$
4. evaluate $P(t)$ and assign fitness
5. while $t \leq T$ do
   - $M(t) = \text{Selection}(P(t))$
   - $Q(t) = \text{Variation}(M(t))$
   - evaluate $Q(t)$
   - $P(t+1) = \text{Survivor}(P(t), Q(t))$
   - $t = t + 1$
6. end while

In RGA, the framework stays familiar. The major redesign is in the **representation** and the **variation operators**.

Real-parameter optimization using EC techniques

Common real-coded optimization techniques include:

- **Real-coded genetic algorithm (RGA)**

Real-parameter optimization using EC techniques

Common real-coded optimization techniques include:

- **Real-coded genetic algorithm (RGA)**
- **Evolution Strategies (ES)**

Real-parameter optimization using EC techniques

Common real-coded optimization techniques include:

- **Real-coded genetic algorithm (RGA)**
- **Evolution Strategies (ES)**
- **Differential Evolution (DE)**

Real-parameter optimization using EC techniques

Common real-coded optimization techniques include:

- **Real-coded genetic algorithm (RGA)**
- **Evolution Strategies (ES)**
- **Differential Evolution (DE)**
- **Particle Swarm Optimization (PSO)**

Real-parameter optimization using EC techniques

Common real-coded optimization techniques include:

- **Real-coded genetic algorithm (RGA)**
- **Evolution Strategies (ES)**
- **Differential Evolution (DE)**
- **Particle Swarm Optimization (PSO)**
- **Artificial Bee Colony (ABC)**

Real-parameter optimization using EC techniques

Common real-coded optimization techniques include:

- **Real-coded genetic algorithm (RGA)**
- **Evolution Strategies (ES)**
- **Differential Evolution (DE)**
- **Particle Swarm Optimization (PSO)**
- **Artificial Bee Colony (ABC)**

RGA is the closest extension of binary GA. It keeps the same overall genetic framework, but operates directly on real-valued variables.

Worked example: Rosenbrock function

We explain the working of RGA through the Rosenbrock function:

$$f(x_1, x_2) = 100(x_2 - x_1^2)^2 + (1 - x_1)^2$$

with bounds

$$-5 \leq x_1 \leq 5, \qquad -5 \leq x_2 \leq 5$$

- Objective: minimize $f(x_1, x_2)$

Worked example: Rosenbrock function

We explain the working of RGA through the Rosenbrock function:

$$f(x_1, x_2) = 100(x_2 - x_1^2)^2 + (1 - x_1)^2$$

with bounds

$$-5 \leq x_1 \leq 5, \qquad -5 \leq x_2 \leq 5$$

- Objective: minimize $f(x_1, x_2)$
- Global optimum: $\boldsymbol{x}^* = (1, 1)^T$

Worked example: Rosenbrock function

We explain the working of RGA through the Rosenbrock function:

$$f(x_1, x_2) = 100(x_2 - x_1^2)^2 + (1 - x_1)^2$$

with bounds

$$-5 \leq x_1 \leq 5, \qquad -5 \leq x_2 \leq 5$$

- Objective: minimize $f(x_1, x_2)$
- Global optimum: $\boldsymbol{x}^* = (1, 1)^T$
- Optimum value: $f(\boldsymbol{x}^*) = 0$

Why Rosenbrock is a useful teaching example

Why this function is important

- continuous search space

Why Rosenbrock is a useful teaching example

Why this function is important

- continuous search space
- curved valley around the optimum region

Why Rosenbrock is a useful teaching example

Why this function is important

- continuous search space
- curved valley around the optimum region
- small numerical changes can matter a lot

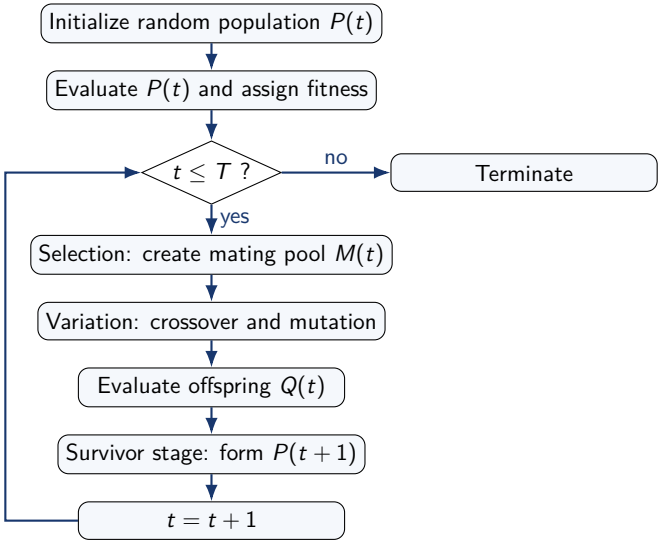
Why Rosenbrock is a useful teaching example

Why this function is important

- continuous search space
- curved valley around the optimum region
- small numerical changes can matter a lot

A good optimizer must not only improve the objective value, but also move carefully inside a sensitive continuous landscape.

Flow of the real-coded GA



Initial population for the example

Let the population size be $N = 8$.

Index	x_1	x_2
1	2.212	3.009
2	-2.289	-2.396
3	-2.933	-4.790
4	-0.639	1.692
5	-3.168	0.706
6	0.215	-2.350
7	-0.742	1.934
8	-4.563	4.791

Initial population for the example

Let the population size be $N = 8$.

Index	x_1	x_2
1	2.212	3.009
2	-2.289	-2.396
3	-2.933	-4.790
4	-0.639	1.692
5	-3.168	0.706
6	0.215	-2.350
7	-0.742	1.934
8	-4.563	4.791

Each row is already a real-valued chromosome. There is no binary string here.

How do we evaluate one solution?

Take solution 4:

$$\boldsymbol{x}^{(4)} = (-0.639,\ 1.692)^T$$

Substitute into the Rosenbrock function:

$$f(x_1, x_2) = 100(x_2 - x_1^2)^2 + (1 - x_1)^2$$

How do we evaluate one solution?

Take solution 4:

$$\boldsymbol{x}^{(4)} = (-0.639,\ 1.692)^T$$

Substitute into the Rosenbrock function:

$$f(x_1, x_2) = 100(x_2 - x_1^2)^2 + (1 - x_1)^2$$

$$f(\boldsymbol{x}^{(4)}) = 100(1.692 - (-0.639)^2)^2 + (1 - (-0.639))^2$$

How do we evaluate one solution?

Take solution 4:

$$\boldsymbol{x}^{(4)} = (-0.639,\ 1.692)^T$$

Substitute into the Rosenbrock function:

$$f(x_1, x_2) = 100(x_2 - x_1^2)^2 + (1 - x_1)^2$$

$$f(\boldsymbol{x}^{(4)}) = 100(1.692 - (-0.639)^2)^2 + (1 - (-0.639))^2$$

$$= 100(1.692 - 0.408321)^2 + (1.639)^2$$

How do we evaluate one solution?

Take solution 4:

$$\boldsymbol{x}^{(4)} = (-0.639,\ 1.692)^T$$

Substitute into the Rosenbrock function:

$$f(x_1, x_2) = 100(x_2 - x_1^2)^2 + (1 - x_1)^2$$

$$f(\boldsymbol{x}^{(4)}) = 100(1.692 - (-0.639)^2)^2 + (1 - (-0.639))^2$$

$$= 100(1.692 - 0.408321)^2 + (1.639)^2$$

$$= 100(1.283679)^2 + 2.686321 \approx 167.469$$

How do we evaluate one solution?

Take solution 4:

$$\boldsymbol{x}^{(4)} = (-0.639,\ 1.692)^T$$

Substitute into the Rosenbrock function:

$$f(x_1, x_2) = 100(x_2 - x_1^2)^2 + (1 - x_1)^2$$

$$f(\boldsymbol{x}^{(4)}) = 100(1.692 - (-0.639)^2)^2 + (1 - (-0.639))^2$$

$$= 100(1.692 - 0.408321)^2 + (1.639)^2$$

$$= 100(1.283679)^2 + 2.686321 \approx 167.469$$

We repeat exactly the same process for all 8 solutions.

Evaluation of the full initial population

Index	x_1	x_2	$f(x_1, x_2)$
1	2.212	3.009	356.393
2	-2.289	-2.396	5840.936
3	-2.933	-4.790	17951.345
4	-0.639	1.692	167.469
5	-3.168	0.706	8722.680
6	0.215	-2.350	574.806
7	-0.742	1.934	194.424
8	-4.563	4.791	25726.938

Interpretation of the evaluation table

Because this is a minimization problem:

- smaller objective value means better solution

Interpretation of the evaluation table

Because this is a minimization problem:

- smaller objective value means better solution
- solution 4 is currently the best parent

Interpretation of the evaluation table

Because this is a minimization problem:

- smaller objective value means better solution
- solution 4 is currently the best parent
- solution 8 is currently the worst parent

Interpretation of the evaluation table

Because this is a minimization problem:

- smaller objective value means better solution
- solution 4 is currently the best parent
- solution 8 is currently the worst parent

At this stage, RGA behaves just like BGA from the selection point of view: evaluate, compare, and prefer better fitness values.

Termination condition

Assume the current generation is $t = 1$ and the maximum generation is T.

- If $t > T$, the algorithm terminates.

Termination condition

Assume the current generation is $t = 1$ and the maximum generation is T.

- If $t > T$, the algorithm terminates.
- Otherwise, we proceed to the selection operator.

Termination condition

Assume the current generation is $t = 1$ and the maximum generation is T.

- If $t > T$, the algorithm terminates.
- Otherwise, we proceed to the selection operator.
- In this lecture, selection is done by binary tournament selection.

Termination condition

Assume the current generation is $t = 1$ and the maximum generation is T.

- If $t > T$, the algorithm terminates.
- Otherwise, we proceed to the selection operator.
- In this lecture, selection is done by binary tournament selection.

Selection itself does not need any special real-coded modification. The major structural changes begin in crossover and mutation.

Binary tournament selection in RGA

1. Randomly choose two candidate solutions.

Binary tournament selection in RGA

1. Randomly choose two candidate solutions.
2. Compare their objective values.

Binary tournament selection in RGA

1. Randomly choose two candidate solutions.
2. Compare their objective values.
3. For a minimization problem, the smaller objective value wins.

Binary tournament selection in RGA

1. Randomly choose two candidate solutions.
2. Compare their objective values.
3. For a minimization problem, the smaller objective value wins.
4. Put the winner into the mating pool.

Binary tournament selection in RGA

1. Randomly choose two candidate solutions.
2. Compare their objective values.
3. For a minimization problem, the smaller objective value wins.
4. Put the winner into the mating pool.
5. Repeat until the mating pool is full.

Binary tournament selection in RGA

1. Randomly choose two candidate solutions.
2. Compare their objective values.
3. For a minimization problem, the smaller objective value wins.
4. Put the winner into the mating pool.
5. Repeat until the mating pool is full.

This is exactly the same selection idea we studied in binary GA. Only the chromosome representation is different.

Binary tournament selection: step-by-step tournaments

Suppose we perform the following 8 tournaments.

Tournament	Candidates	Objective values	Winner	Reason
1	$(7, 6)$	(194.424, 574.806)	7	smaller value
2	$(4, 5)$	(167.469, 8722.680)	4	smaller value
3	$(8, 3)$	(25726.938, 17951.345)	3	smaller value
4	$(1, 2)$	(356.393, 5840.936)	1	smaller value
5	$(3, 1)$	(17951.345, 356.393)	1	smaller value
6	$(2, 8)$	(5840.936, 25726.938)	2	smaller value
7	$(7, 5)$	(194.424, 8722.680)	7	smaller value
8	$(4, 6)$	(167.469, 574.806)	4	smaller value

How the mating pool is formed from the tournaments

The winners of the 8 tournaments are:

$$7,\ 4,\ 3,\ 1,\ 1,\ 2,\ 7,\ 4$$

So the mating pool becomes:

New index	Original winner	(x_1, x_2)	$f(x_1, x_2)$
1	7	$(-0.742,\ 1.934)$	194.424
2	4	$(-0.639,\ 1.692)$	167.469
3	3	$(-2.933,\ -4.790)$	17951.345
4	1	$(2.212,\ 3.009)$	356.393
5	1	$(2.212,\ 3.009)$	356.393
6	2	$(-2.289,\ -2.396)$	5840.936
7	7	$(-0.742,\ 1.934)$	194.424
8	4	$(-0.639,\ 1.692)$	167.469

How the mating pool is formed from the tournaments

The winners of the 8 tournaments are:

$$7,\ 4,\ 3,\ 1,\ 1,\ 2,\ 7,\ 4$$

So the mating pool becomes:

New index	Original winner	(x_1, x_2)	$f(x_1, x_2)$
1	7	$(-0.742,\ 1.934)$	194.424
2	4	$(-0.639,\ 1.692)$	167.469
3	3	$(-2.933,\ -4.790)$	17951.345
4	1	$(2.212,\ 3.009)$	356.393
5	1	$(2.212,\ 3.009)$	356.393
6	2	$(-2.289,\ -2.396)$	5840.936
7	7	$(-0.742,\ 1.934)$	194.424
8	4	$(-0.639,\ 1.692)$	167.469

Good solutions can appear multiple times in the mating pool. Weak solutions may disappear completely.

From the mating pool to crossover pairs

Now we randomly pair the solutions in the mating pool. Suppose the pair order is:

$$(3, 7), \qquad (5, 2), \qquad (8, 4), \qquad (6, 1)$$

- These are **mating-pool indices**, not original parent indices.

From the mating pool to crossover pairs

Now we randomly pair the solutions in the mating pool. Suppose the pair order is:

$$(3, 7), \qquad (5, 2), \qquad (8, 4), \qquad (6, 1)$$

- These are **mating-pool indices**, not original parent indices.
- Each pair will decide whether crossover happens.

From the mating pool to crossover pairs

Now we randomly pair the solutions in the mating pool. Suppose the pair order is:

$$(3, 7), \qquad (5, 2), \qquad (8, 4), \qquad (6, 1)$$

- These are **mating-pool indices**, not original parent indices.
- Each pair will decide whether crossover happens.
- If crossover does not happen, the pair is copied unchanged.

From the mating pool to crossover pairs

Now we randomly pair the solutions in the mating pool. Suppose the pair order is:

$$(3,7), \qquad (5,2), \qquad (8,4), \qquad (6,1)$$

- These are **mating-pool indices**, not original parent indices.
- Each pair will decide whether crossover happens.
- If crossover does not happen, the pair is copied unchanged.

This is one place where students often get confused. After selection, we usually relabel the mating pool from 1 to N and then perform pairing on these new positions.

Why single-point crossover is not suitable here

In binary GA, single-point crossover works on bit strings.

- RGA chromosomes are real-valued vectors.

Why single-point crossover is not suitable here

In binary GA, single-point crossover works on bit strings.

- RGA chromosomes are real-valued vectors.
- We therefore need a crossover operator that acts directly on real numbers.

Why single-point crossover is not suitable here

In binary GA, single-point crossover works on bit strings.

- RGA chromosomes are real-valued vectors.
- We therefore need a crossover operator that acts directly on real numbers.
- At the same time, we want it to preserve useful genetic behavior.

Why single-point crossover is not suitable here

In binary GA, single-point crossover works on bit strings.

- RGA chromosomes are real-valued vectors.
- We therefore need a crossover operator that acts directly on real numbers.
- At the same time, we want it to preserve useful genetic behavior.

A widely used real-valued crossover operator is **Simulated Binary Crossover (SBX)**.

Property 1 from binary GA: average is preserved

Consider a one-point crossover example in binary GA:

	Parent strings	Decoded value
Parent 1	101001	41
Parent 2	011010	26

Property 1 from binary GA: average is preserved

Consider a one-point crossover example in binary GA:

	Parent strings	Decoded value
Parent 1	101001	41
Parent 2	011010	26

Average of parent values:

Property 1 from binary GA: average is preserved

Consider a one-point crossover example in binary GA:

	Parent strings	Decoded value
Parent 1	101001	41
Parent 2	011010	26

Average of parent values:

$$\frac{41 + 26}{2} = 33.5$$

Property 1 from binary GA: average is preserved

Consider a one-point crossover example in binary GA:

	Parent strings	Decoded value
Parent 1	101001	41
Parent 2	011010	26

Average of parent values:

$$\frac{41 + 26}{2} = 33.5$$

After crossover, suppose the offspring are:

Property 1 from binary GA: average is preserved

Consider a one-point crossover example in binary GA:

	Parent strings	Decoded value
Parent 1	101001	41
Parent 2	011010	26

Average of parent values:

$$\frac{41 + 26}{2} = 33.5$$

After crossover, suppose the offspring are:

	Offspring strings	Decoded value
Offspring 1	101010	42
Offspring 2	011001	25

Property 1 from binary GA: average is preserved

Consider a one-point crossover example in binary GA:

	Parent strings	Decoded value
Parent 1	101001	41
Parent 2	011010	26

Average of parent values:

$$\frac{41 + 26}{2} = 33.5$$

After crossover, suppose the offspring are:

	Offspring strings	Decoded value
Offspring 1	101010	42
Offspring 2	011001	25

Average of offspring values:

Property 1 from binary GA: average is preserved

Consider a one-point crossover example in binary GA:

	Parent strings	Decoded value
Parent 1	101001	41
Parent 2	011010	26

Average of parent values:

$$\frac{41 + 26}{2} = 33.5$$

After crossover, suppose the offspring are:

	Offspring strings	Decoded value
Offspring 1	101010	42
Offspring 2	011001	25

Average of offspring values:

$$\frac{42 + 25}{2} = 33.5$$

Property 1 from binary GA: average is preserved

Consider a one-point crossover example in binary GA:

	Parent strings	Decoded value
Parent 1	101001	41
Parent 2	011010	26

Average of parent values:

$$\frac{41 + 26}{2} = 33.5$$

After crossover, suppose the offspring are:

	Offspring strings	Decoded value
Offspring 1	101010	42
Offspring 2	011001	25

Average of offspring values:

$$\frac{42 + 25}{2} = 33.5$$

So the average before and after crossover is the same. This is one important behavior that SBX tries to mimic in continuous space.

Property 2 from binary GA: spread factor

A second useful idea is how far apart the offspring are compared with the parents.

Property 2 from binary GA: spread factor

A second useful idea is how far apart the offspring are compared with the parents.
Using the same decoded values:

Property 2 from binary GA: spread factor

A second useful idea is how far apart the offspring are compared with the parents.
Using the same decoded values:

$$\text{parent spread} = |41 - 26| = 15$$

$$\text{offspring spread} = |42 - 25| = 17$$

Property 2 from binary GA: spread factor

A second useful idea is how far apart the offspring are compared with the parents.
Using the same decoded values:

$$\text{parent spread} = |41 - 26| = 15$$

$$\text{offspring spread} = |42 - 25| = 17$$

Define the spread factor as

Property 2 from binary GA: spread factor

A second useful idea is how far apart the offspring are compared with the parents. Using the same decoded values:

$$\text{parent spread} = |41 - 26| = 15$$

$$\text{offspring spread} = |42 - 25| = 17$$

Define the spread factor as

$$\beta = \frac{|o_2 - o_1|}{|p_2 - p_1|}$$

Property 2 from binary GA: spread factor

A second useful idea is how far apart the offspring are compared with the parents.
Using the same decoded values:

$$\text{parent spread} = |41 - 26| = 15$$

$$\text{offspring spread} = |42 - 25| = 17$$

Define the spread factor as

$$\beta = \frac{|o_2 - o_1|}{|p_2 - p_1|}$$

For this example,

Property 2 from binary GA: spread factor

A second useful idea is how far apart the offspring are compared with the parents. Using the same decoded values:

$$\text{parent spread} = |41 - 26| = 15$$

$$\text{offspring spread} = |42 - 25| = 17$$

Define the spread factor as

$$\beta = \frac{|o_2 - o_1|}{|p_2 - p_1|}$$

For this example,

$$\beta = \frac{17}{15} \approx 1.13$$

Property 2 from binary GA: spread factor

A second useful idea is how far apart the offspring are compared with the parents.
Using the same decoded values:

$$\text{parent spread} = |41 - 26| = 15$$

$$\text{offspring spread} = |42 - 25| = 17$$

Define the spread factor as

$$\beta = \frac{|o_2 - o_1|}{|p_2 - p_1|}$$

For this example,

$$\beta = \frac{17}{15} \approx 1.13$$

- $\beta < 1$: contracting crossover, offspring come closer together
- $\beta = 1$: stationary crossover, spread stays the same
- $\beta > 1$: expanding crossover, offspring move farther apart

Why these properties matter for SBX

SBX was designed so that a real-valued crossover behaves similarly to binary one-point crossover.

- It preserves the **average** of the two parent values.

Why these properties matter for SBX

SBX was designed so that a real-valued crossover behaves similarly to binary one-point crossover.

- It preserves the **average** of the two parent values.
- It controls the **spread** of offspring through the factor β.

Why these properties matter for SBX

SBX was designed so that a real-valued crossover behaves similarly to binary one-point crossover.

- It preserves the **average** of the two parent values.
- It controls the **spread** of offspring through the factor β.
- This allows the operator to either search near the parents or explore beyond them.

Why these properties matter for SBX

SBX was designed so that a real-valued crossover behaves similarly to binary one-point crossover.

- It preserves the **average** of the two parent values.
- It controls the **spread** of offspring through the factor β.
- This allows the operator to either search near the parents or explore beyond them.

These two ideas - midpoint preservation and controlled spread - are the bridge from BGA crossover to real-coded crossover.

SBX formulas

For one variable, assume $p_1 < p_2$. SBX generates two offspring as:

$$o_1 = \bar{p} - \frac{1}{2}\beta(p_2 - p_1), \qquad o_2 = \bar{p} + \frac{1}{2}\beta(p_2 - p_1)$$

where

$$\bar{p} = \frac{p_1 + p_2}{2}$$

- If $\beta < 1$, offspring move closer together.

SBX formulas

For one variable, assume $p_1 < p_2$. SBX generates two offspring as:

$$o_1 = \bar{p} - \frac{1}{2}\beta(p_2 - p_1), \qquad o_2 = \bar{p} + \frac{1}{2}\beta(p_2 - p_1)$$

where

$$\bar{p} = \frac{p_1 + p_2}{2}$$

- If $\beta < 1$, offspring move closer together.
- If $\beta > 1$, offspring spread farther apart.

SBX formulas

For one variable, assume $p_1 < p_2$. SBX generates two offspring as:

$$o_1 = \bar{p} - \frac{1}{2}\beta(p_2 - p_1), \qquad o_2 = \bar{p} + \frac{1}{2}\beta(p_2 - p_1)$$

where

$$\bar{p} = \frac{p_1 + p_2}{2}$$

- If $\beta < 1$, offspring move closer together.
- If $\beta > 1$, offspring spread farther apart.
- If $\beta = 1$, offspring spacing equals parent spacing.

How is the spread factor chosen in SBX?

The spread factor for the ith variable is sampled from a probability distribution controlled by η_c:

$$\beta_i = \begin{cases} (2u_i)^{\frac{1}{\eta_c+1}}, & u_i \leq 0.5 \\ \left(\frac{1}{2(1-u_i)}\right)^{\frac{1}{\eta_c+1}}, & u_i > 0.5 \end{cases}$$

where $u_i \in [0, 1]$ is a uniform random number.

- If $u_i \leq 0.5$, then $\beta_i \leq 1$, so the crossover is contracting.

How is the spread factor chosen in SBX?

The spread factor for the ith variable is sampled from a probability distribution controlled by η_c:

$$\beta_i = \begin{cases} (2u_i)^{\frac{1}{\eta_c+1}}, & u_i \leq 0.5 \\ \left(\frac{1}{2(1-u_i)}\right)^{\frac{1}{\eta_c+1}}, & u_i > 0.5 \end{cases}$$

where $u_i \in [0, 1]$ is a uniform random number.

- If $u_i \leq 0.5$, then $\beta_i \leq 1$, so the crossover is contracting.
- If $u_i > 0.5$, then $\beta_i > 1$, so the crossover is expanding.

How is the spread factor chosen in SBX?

The spread factor for the ith variable is sampled from a probability distribution controlled by η_c:

$$\beta_i = \begin{cases} (2u_i)^{\frac{1}{\eta_c+1}}, & u_i \leq 0.5 \\ \left(\frac{1}{2(1-u_i)}\right)^{\frac{1}{\eta_c+1}}, & u_i > 0.5 \end{cases}$$

where $u_i \in [0, 1]$ is a uniform random number.

- If $u_i \leq 0.5$, then $\beta_i \leq 1$, so the crossover is contracting.
- If $u_i > 0.5$, then $\beta_i > 1$, so the crossover is expanding.
- The value of η_c controls how strongly the distribution concentrates near $\beta = 1$.

How to interpret the SBX distribution

- The distribution is centered around the idea that values near $\beta = 1$ are common.

How to interpret the SBX distribution

- The distribution is centered around the idea that values near $\beta = 1$ are common.
- So offspring are often produced near the parent spread.

Interpretation

- $\beta \approx 1$: offspring are near the parents
- very small β: offspring come much closer together
- large β: offspring move beyond the parents

How to interpret the SBX distribution

- The distribution is centered around the idea that values near $\beta = 1$ are common.
- So offspring are often produced near the parent spread.
- But the operator still allows both contraction ($\beta < 1$) and expansion ($\beta > 1$).

Interpretation

- $\beta \approx 1$: offspring are near the parents
- very small β: offspring come much closer together
- large β: offspring move beyond the parents

How to interpret the SBX distribution

- The distribution is centered around the idea that values near $\beta = 1$ are common.
- So offspring are often produced near the parent spread.
- But the operator still allows both contraction ($\beta < 1$) and expansion ($\beta > 1$).
- This gives a balance between local search and exploration.

Interpretation

- $\beta \approx 1$: offspring are near the parents
- very small β: offspring come much closer together
- large β: offspring move beyond the parents

Role of the SBX distribution index η_c

- **Large η_c:** the distribution becomes sharply concentrated near $\beta = 1$.

Role of the SBX distribution index η_c

- **Large η_c:** the distribution becomes sharply concentrated near $\beta = 1$.
- This means offspring are usually close to their parents.

Role of the SBX distribution index η_c

- **Large η_c:** the distribution becomes sharply concentrated near $\beta = 1$.
- This means offspring are usually close to their parents.
- So large η_c gives more **exploitation** or fine local search.

Role of the SBX distribution index η_c

- **Large η_c:** the distribution becomes sharply concentrated near $\beta = 1$.
- This means offspring are usually close to their parents.
- So large η_c gives more **exploitation** or fine local search.
- **Small η_c:** the distribution is flatter and allows more distant offspring.

Role of the SBX distribution index η_c

- **Large η_c:** the distribution becomes sharply concentrated near $\beta = 1$.
- This means offspring are usually close to their parents.
- So large η_c gives more **exploitation** or fine local search.
- **Small η_c:** the distribution is flatter and allows more distant offspring.
- So small η_c gives more **exploration**.

Role of the SBX distribution index η_c

- **Large η_c:** the distribution becomes sharply concentrated near $\beta = 1$.
- This means offspring are usually close to their parents.
- So large η_c gives more **exploitation** or fine local search.
- **Small η_c:** the distribution is flatter and allows more distant offspring.
- So small η_c gives more **exploration**.

Note: *SBX uses η_c to decide how conservative or adventurous the crossover should be.*

Crossover settings for this example

Assume the following crossover settings:

- crossover probability $p_c = 0.9$

Crossover settings for this example

Assume the following crossover settings:

- crossover probability $p_c = 0.9$
- SBX distribution index $\eta_c = 15$

Crossover settings for this example

Assume the following crossover settings:

- crossover probability $p_c = 0.9$
- SBX distribution index $\eta_c = 15$
- random numbers for pairwise crossover decision:

Crossover settings for this example

Assume the following crossover settings:

- crossover probability $p_c = 0.9$
- SBX distribution index $\eta_c = 15$
- random numbers for pairwise crossover decision:

Pair	Random number r
$(3, 7)$	0.63
$(5, 2)$	0.13
$(8, 4)$	0.98
$(6, 1)$	0.57

Crossover settings for this example

Assume the following crossover settings:

- crossover probability $p_c = 0.9$
- SBX distribution index $\eta_c = 15$
- random numbers for pairwise crossover decision:

Pair	Random number r
$(3, 7)$	0.63
$(5, 2)$	0.13
$(8, 4)$	0.98
$(6, 1)$	0.57

- If $r \leq p_c$, crossover is applied.
- If $r > p_c$, the pair is copied unchanged.

Decision: which pairs actually crossover?

Using $p_c = 0.9$:

- Pair $(3, 7)$: $0.63 < 0.9 \Rightarrow$ crossover happens

Decision: which pairs actually crossover?

Using $p_c = 0.9$:

- Pair $(3, 7)$: $0.63 < 0.9 \Rightarrow$ crossover happens
- Pair $(5, 2)$: $0.13 < 0.9 \Rightarrow$ crossover happens

Decision: which pairs actually crossover?

Using $p_c = 0.9$:

- Pair $(3, 7)$: $0.63 < 0.9 \Rightarrow$ crossover happens
- Pair $(5, 2)$: $0.13 < 0.9 \Rightarrow$ crossover happens
- Pair $(8, 4)$: $0.98 > 0.9 \Rightarrow$ copy the pair unchanged

Decision: which pairs actually crossover?

Using $p_c = 0.9$:

- Pair $(3, 7)$: $0.63 < 0.9 \Rightarrow$ crossover happens
- Pair $(5, 2)$: $0.13 < 0.9 \Rightarrow$ crossover happens
- Pair $(8, 4)$: $0.98 > 0.9 \Rightarrow$ copy the pair unchanged
- Pair $(6, 1)$: $0.57 < 0.9 \Rightarrow$ crossover happens

Decision: which pairs actually crossover?

Using $p_c = 0.9$:

- Pair $(3, 7)$: $0.63 < 0.9 \Rightarrow$ crossover happens
- Pair $(5, 2)$: $0.13 < 0.9 \Rightarrow$ crossover happens
- Pair $(8, 4)$: $0.98 > 0.9 \Rightarrow$ copy the pair unchanged
- Pair $(6, 1)$: $0.57 < 0.9 \Rightarrow$ crossover happens

This decision step is important. We do not jump directly from the mating pool to the offspring table.

Detailed SBX calculation for pair $(3, 7)$: variable x_1

From the mating pool:

$$\text{pair } (3,7): \quad (-2.933, -4.790)^T \text{ and } (-0.742, 1.934)^T$$

For variable x_1:

$$p_1 = -2.933, \qquad p_2 = -0.742$$

Detailed SBX calculation for pair $(3, 7)$: variable x_1

From the mating pool:

$$\text{pair } (3,7): \quad (-2.933, -4.790)^T \text{ and } (-0.742, 1.934)^T$$

For variable x_1:

$$p_1 = -2.933, \qquad p_2 = -0.742$$

Assume $u_1 = 0.206$. Then

Detailed SBX calculation for pair $(3, 7)$: variable x_1

From the mating pool:

$$\text{pair } (3,7): \quad (-2.933, -4.790)^T \text{ and } (-0.742, 1.934)^T$$

For variable x_1:

$$p_1 = -2.933, \qquad p_2 = -0.742$$

Assume $u_1 = 0.206$. Then

$$\beta_1 = (2u_1)^{\frac{1}{\eta_c+1}} = (0.412)^{\frac{1}{16}} \approx 0.946$$

Detailed SBX calculation for pair $(3, 7)$: variable x_1

From the mating pool:

$$\text{pair } (3,7): \quad (-2.933, -4.790)^T \text{ and } (-0.742, 1.934)^T$$

For variable x_1:

$$p_1 = -2.933, \qquad p_2 = -0.742$$

Assume $u_1 = 0.206$. Then

$$\beta_1 = (2u_1)^{\frac{1}{\eta_c+1}} = (0.412)^{\frac{1}{16}} \approx 0.946$$

The midpoint is

Detailed SBX calculation for pair $(3, 7)$: variable x_1

From the mating pool:

$$\text{pair } (3,7): \quad (-2.933, -4.790)^T \text{ and } (-0.742, 1.934)^T$$

For variable x_1:

$$p_1 = -2.933, \qquad p_2 = -0.742$$

Assume $u_1 = 0.206$. Then

$$\beta_1 = (2u_1)^{\frac{1}{\eta_c+1}} = (0.412)^{\frac{1}{16}} \approx 0.946$$

The midpoint is

$$\bar{p} = \frac{-2.933 + (-0.742)}{2} = -1.8375$$

Detailed SBX calculation for pair $(3, 7)$: variable x_1

From the mating pool:

$$\text{pair } (3,7): \quad (-2.933, -4.790)^T \text{ and } (-0.742, 1.934)^T$$

For variable x_1:

$$p_1 = -2.933, \qquad p_2 = -0.742$$

Assume $u_1 = 0.206$. Then

$$\beta_1 = (2u_1)^{\frac{1}{\eta_c+1}} = (0.412)^{\frac{1}{16}} \approx 0.946$$

The midpoint is

$$\bar{p} = \frac{-2.933 + (-0.742)}{2} = -1.8375$$

Now compute the offspring values:

Detailed SBX calculation for pair $(3, 7)$: variable x_1

From the mating pool:

$$\text{pair } (3,7): \quad (-2.933, -4.790)^T \text{ and } (-0.742, 1.934)^T$$

For variable x_1:

$$p_1 = -2.933, \qquad p_2 = -0.742$$

Assume $u_1 = 0.206$. Then

$$\beta_1 = (2u_1)^{\frac{1}{\eta_c+1}} = (0.412)^{\frac{1}{16}} \approx 0.946$$

The midpoint is

$$\bar{p} = \frac{-2.933 + (-0.742)}{2} = -1.8375$$

Now compute the offspring values:

$$o_1 = -1.8375 - \frac{1}{2}(0.946)(2.191) \approx -2.874$$

Detailed SBX calculation for pair $(3, 7)$: variable x_1

From the mating pool:

$$\text{pair } (3,7): \quad (-2.933, -4.790)^T \text{ and } (-0.742, 1.934)^T$$

For variable x_1:

$$p_1 = -2.933, \qquad p_2 = -0.742$$

Assume $u_1 = 0.206$. Then

$$\beta_1 = (2u_1)^{\frac{1}{\eta_c+1}} = (0.412)^{\frac{1}{16}} \approx 0.946$$

The midpoint is

$$\bar{p} = \frac{-2.933 + (-0.742)}{2} = -1.8375$$

Now compute the offspring values:

$$o_1 = -1.8375 - \frac{1}{2}(0.946)(2.191) \approx -2.874$$

$$o_2 = -1.8375 + \frac{1}{2}(0.946)(2.191) \approx -0.801$$

Detailed SBX calculation for pair $(3, 7)$: variable x_2

For variable x_2 of the same pair:

$$p_1 = -4.790, \qquad p_2 = 1.934$$

Detailed SBX calculation for pair $(3, 7)$: variable x_2

For variable x_2 of the same pair:

$$p_1 = -4.790, \qquad p_2 = 1.934$$

Assume $u_2 = 0.461$. Then

Detailed SBX calculation for pair $(3, 7)$: variable x_2

For variable x_2 of the same pair:

$$p_1 = -4.790, \qquad p_2 = 1.934$$

Assume $u_2 = 0.461$. Then

$$\beta_2 = (2u_2)^{\frac{1}{16}} = (0.922)^{\frac{1}{16}} \approx 0.995$$

Detailed SBX calculation for pair $(3, 7)$: variable x_2

For variable x_2 of the same pair:

$$p_1 = -4.790, \qquad p_2 = 1.934$$

Assume $u_2 = 0.461$. Then

$$\beta_2 = (2u_2)^{\frac{1}{16}} = (0.922)^{\frac{1}{16}} \approx 0.995$$

The midpoint is

Detailed SBX calculation for pair $(3, 7)$: variable x_2

For variable x_2 of the same pair:

$$p_1 = -4.790, \qquad p_2 = 1.934$$

Assume $u_2 = 0.461$. Then

$$\beta_2 = (2u_2)^{\frac{1}{16}} = (0.922)^{\frac{1}{16}} \approx 0.995$$

The midpoint is

$$\bar{p} = \frac{-4.790 + 1.934}{2} = -1.428$$

Detailed SBX calculation for pair $(3, 7)$: variable x_2

For variable x_2 of the same pair:

$$p_1 = -4.790, \qquad p_2 = 1.934$$

Assume $u_2 = 0.461$. Then

$$\beta_2 = (2u_2)^{\frac{1}{16}} = (0.922)^{\frac{1}{16}} \approx 0.995$$

The midpoint is

$$\bar{p} = \frac{-4.790 + 1.934}{2} = -1.428$$

Now compute the offspring values:

Detailed SBX calculation for pair $(3, 7)$: variable x_2

For variable x_2 of the same pair:

$$p_1 = -4.790, \qquad p_2 = 1.934$$

Assume $u_2 = 0.461$. Then

$$\beta_2 = (2u_2)^{\frac{1}{16}} = (0.922)^{\frac{1}{16}} \approx 0.995$$

The midpoint is

$$\bar{p} = \frac{-4.790 + 1.934}{2} = -1.428$$

Now compute the offspring values:

$$o_1 = -1.428 - \frac{1}{2}(0.995)(6.724) \approx -4.773$$

Detailed SBX calculation for pair $(3, 7)$: variable x_2

For variable x_2 of the same pair:

$$p_1 = -4.790, \qquad p_2 = 1.934$$

Assume $u_2 = 0.461$. Then

$$\beta_2 = (2u_2)^{\frac{1}{16}} = (0.922)^{\frac{1}{16}} \approx 0.995$$

The midpoint is

$$\bar{p} = \frac{-4.790 + 1.934}{2} = -1.428$$

Now compute the offspring values:

$$o_1 = -1.428 - \frac{1}{2}(0.995)(6.724) \approx -4.773$$

$$o_2 = -1.428 + \frac{1}{2}(0.995)(6.724) \approx 1.917$$

Detailed SBX calculation for pair $(3, 7)$: variable x_2

For variable x_2 of the same pair:

$$p_1 = -4.790, \qquad p_2 = 1.934$$

Assume $u_2 = 0.461$. Then

$$\beta_2 = (2u_2)^{\frac{1}{16}} = (0.922)^{\frac{1}{16}} \approx 0.995$$

The midpoint is

$$\bar{p} = \frac{-4.790 + 1.934}{2} = -1.428$$

Now compute the offspring values:

$$o_1 = -1.428 - \frac{1}{2}(0.995)(6.724) \approx -4.773$$

$$o_2 = -1.428 + \frac{1}{2}(0.995)(6.724) \approx 1.917$$

So the two children from pair $(3, 7)$ are approximately $(-2.874, -4.773)^T$ and $(-0.801, 1.917)^T$.

Summary of the remaining crossover calculations

Pair $(5, 2)$

- Parents: $(2.212, 3.009)^T$ and $(-0.639, 1.692)^T$

Summary of the remaining crossover calculations

Pair $(5, 2)$

- Parents: $(2.212, 3.009)^T$ and $(-0.639, 1.692)^T$
- Using $u_1 = 0.890$ and $u_2 = 0.511$, we get approximately:

$$(-0.780, 1.691)^T \quad \text{and} \quad (2.353, 3.010)^T$$

Summary of the remaining crossover calculations

Pair $(5, 2)$

- Parents: $(2.212, 3.009)^T$ and $(-0.639, 1.692)^T$
- Using $u_1 = 0.890$ and $u_2 = 0.511$, we get approximately:

$$(-0.780, 1.691)^T \quad \text{and} \quad (2.353, 3.010)^T$$

Summary of the remaining crossover calculations

Pair $(5, 2)$

- Parents: $(2.212, 3.009)^T$ and $(-0.639, 1.692)^T$
- Using $u_1 = 0.890$ and $u_2 = 0.511$, we get approximately:

$$(-0.780, 1.691)^T \quad \text{and} \quad (2.353, 3.010)^T$$

Pair $(8, 4)$

Summary of the remaining crossover calculations

Pair $(5, 2)$

- Parents: $(2.212, 3.009)^T$ and $(-0.639, 1.692)^T$
- Using $u_1 = 0.890$ and $u_2 = 0.511$, we get approximately:

$$(-0.780, 1.691)^T \quad \text{and} \quad (2.353, 3.010)^T$$

Pair $(8, 4)$

- Since $r = 0.98 > 0.9$, no crossover is performed.
- The pair is copied as $(-0.639, 1.692)^T$ and $(2.212, 3.009)^T$.

Summary of the remaining crossover calculations

Pair $(5, 2)$

- Parents: $(2.212, 3.009)^T$ and $(-0.639, 1.692)^T$
- Using $u_1 = 0.890$ and $u_2 = 0.511$, we get approximately:

$$(-0.780, 1.691)^T \quad \text{and} \quad (2.353, 3.010)^T$$

Pair $(8, 4)$

- Since $r = 0.98 > 0.9$, no crossover is performed.
- The pair is copied as $(-0.639, 1.692)^T$ and $(2.212, 3.009)^T$.

Summary of the remaining crossover calculations

Pair $(5, 2)$

- Parents: $(2.212, 3.009)^T$ and $(-0.639, 1.692)^T$
- Using $u_1 = 0.890$ and $u_2 = 0.511$, we get approximately:

$$(-0.780, 1.691)^T \quad \text{and} \quad (2.353, 3.010)^T$$

Pair $(8, 4)$

- Since $r = 0.98 > 0.9$, no crossover is performed.
- The pair is copied as $(-0.639, 1.692)^T$ and $(2.212, 3.009)^T$.

Summary of the remaining crossover calculations

Pair $(5, 2)$

- Parents: $(2.212, 3.009)^T$ and $(-0.639, 1.692)^T$
- Using $u_1 = 0.890$ and $u_2 = 0.511$, we get approximately:

$$(-0.780, 1.691)^T \quad \text{and} \quad (2.353, 3.010)^T$$

Pair $(8, 4)$

- Since $r = 0.98 > 0.9$, no crossover is performed.
- The pair is copied as $(-0.639, 1.692)^T$ and $(2.212, 3.009)^T$.

Pair $(6, 1)$

Summary of the remaining crossover calculations

Pair $(5, 2)$

- Parents: $(2.212, 3.009)^T$ and $(-0.639, 1.692)^T$
- Using $u_1 = 0.890$ and $u_2 = 0.511$, we get approximately:

$$(-0.780, 1.691)^T \quad \text{and} \quad (2.353, 3.010)^T$$

Pair $(8, 4)$

- Since $r = 0.98 > 0.9$, no crossover is performed.
- The pair is copied as $(-0.639, 1.692)^T$ and $(2.212, 3.009)^T$.

Pair $(6, 1)$

- Using $\beta_1 \approx 0.914$ and $\beta_2 \approx 0.975$, the offspring are approximately:

$$(-2.222, -2.342)^T \quad \text{and} \quad (-0.809, 1.880)^T$$

Offspring population after crossover

Offspring	x_1	x_2	$f(x_1, x_2)$
1	-2.874	-4.773	17000.594
2	-0.801	1.917	165.908
3	-0.780	1.691	120.371
4	2.353	3.010	640.206
5	-0.639	1.692	167.469
6	2.212	3.009	356.393
7	-2.222	-2.342	5309.179
8	-0.809	1.880	153.462

Offspring population after crossover

Offspring	x_1	x_2	$f(x_1, x_2)$
1	-2.874	-4.773	17000.594
2	-0.801	1.917	165.908
3	-0.780	1.691	120.371
4	2.353	3.010	640.206
5	-0.639	1.692	167.469
6	2.212	3.009	356.393
7	-2.222	-2.342	5309.179
8	-0.809	1.880	153.462

Crossover can produce both better and worse solutions. Here offspring 3 and offspring 8 are already better than the best parent.

What students should notice after crossover

- Crossover recombines the information present in the selected parents.

What students should notice after crossover

- Crossover recombines the information present in the selected parents.
- It does not guarantee that every child is better.

What students should notice after crossover

- Crossover recombines the information present in the selected parents.
- It does not guarantee that every child is better.
- But it can create children better than all parents seen so far.

What students should notice after crossover

- Crossover recombines the information present in the selected parents.
- It does not guarantee that every child is better.
- But it can create children better than all parents seen so far.

This is why the selection operator and the variation operator play different roles: selection chooses promising parents, while crossover tries to create promising new combinations.

Polynomial mutation for real numbers

In binary GA, mutation flips bits. In RGA, mutation perturbs real values directly.

Polynomial mutation for real numbers

In binary GA, mutation flips bits. In RGA, mutation perturbs real values directly.
A common operator is **polynomial mutation**.

Polynomial mutation for real numbers

In binary GA, mutation flips bits. In RGA, mutation perturbs real values directly.
A common operator is **polynomial mutation**.
A common update rule is

Polynomial mutation for real numbers

In binary GA, mutation flips bits. In RGA, mutation perturbs real values directly.
A common operator is **polynomial mutation**.
A common update rule is

$$y_i^{(t+1)} = x_i^{(t+1)} + \left(x_i^{(U)} - x_i^{(L)}\right) \delta_i$$

Polynomial mutation for real numbers

In binary GA, mutation flips bits. In RGA, mutation perturbs real values directly.
A common operator is **polynomial mutation**.
A common update rule is

$$y_i^{(t+1)} = x_i^{(t+1)} + \left(x_i^{(U)} - x_i^{(L)}\right) \delta_i$$

where δ_i is generated from a polynomial-shaped distribution.

Polynomial mutation for real numbers

In binary GA, mutation flips bits. In RGA, mutation perturbs real values directly.
A common operator is **polynomial mutation**.
A common update rule is

$$y_i^{(t+1)} = x_i^{(t+1)} + \left(x_i^{(U)} - x_i^{(L)}\right) \delta_i$$

where δ_i is generated from a polynomial-shaped distribution.

Mutation is usually applied with a small probability. It provides local perturbation and extra diversity.

How the mutation perturbation is generated

For polynomial mutation, one common expression is:

$$\delta_i = \begin{cases} (2r_i)^{\frac{1}{\eta_m+1}} - 1, & r_i < 0.5 \\ 1 - [2(1-r_i)]^{\frac{1}{\eta_m+1}}, & r_i \geq 0.5 \end{cases}$$

where $r_i \in [0, 1]$ is random and η_m is the mutation distribution index.

- δ_i always lies between -1 and 1.

How the mutation perturbation is generated

For polynomial mutation, one common expression is:

$$\delta_i = \begin{cases} (2r_i)^{\frac{1}{\eta_m+1}} - 1, & r_i < 0.5 \\ 1 - [2(1-r_i)]^{\frac{1}{\eta_m+1}}, & r_i \geq 0.5 \end{cases}$$

where $r_i \in [0, 1]$ is random and η_m is the mutation distribution index.

- δ_i always lies between -1 and 1.
- If $r_i < 0.5$, then $\delta_i < 0$ and the variable moves downward.

How the mutation perturbation is generated

For polynomial mutation, one common expression is:

$$\delta_i = \begin{cases} (2r_i)^{\frac{1}{\eta_m+1}} - 1, & r_i < 0.5 \\ 1 - [2(1-r_i)]^{\frac{1}{\eta_m+1}}, & r_i \geq 0.5 \end{cases}$$

where $r_i \in [0, 1]$ is random and η_m is the mutation distribution index.

- δ_i always lies between -1 and 1.
- If $r_i < 0.5$, then $\delta_i < 0$ and the variable moves downward.
- If $r_i \geq 0.5$, then $\delta_i \geq 0$ and the variable moves upward.

How to interpret the mutation distribution

- The distribution is symmetric around $\delta = 0$.

How to interpret the mutation distribution

- The distribution is symmetric around $\delta = 0$.
- Values close to $\delta = 0$ mean small perturbations.

How to interpret the mutation distribution

- The distribution is symmetric around $\delta = 0$.
- Values close to $\delta = 0$ mean small perturbations.
- Values near -1 or $+1$ mean large perturbations.

How to interpret the mutation distribution

- The distribution is symmetric around $\delta = 0$.
- Values close to $\delta = 0$ mean small perturbations.
- Values near -1 or $+1$ mean large perturbations.
- In practice, most mutations are small, while large jumps are less frequent.

How to interpret the mutation distribution

- The distribution is symmetric around $\delta = 0$.
- Values close to $\delta = 0$ mean small perturbations.
- Values near -1 or $+1$ mean large perturbations.
- In practice, most mutations are small, while large jumps are less frequent.

This is why polynomial mutation is useful: it usually performs local refinement, but it still keeps a chance of making a larger move.

Role of the mutation distribution index η_m

- **Large η_m:** the distribution concentrates near $\delta = 0$.

Role of the mutation distribution index η_m

- **Large η_m:** the distribution concentrates near $\delta = 0$.
- So the mutation becomes gentle and local.

Role of the mutation distribution index η_m

- **Large η_m:** the distribution concentrates near $\delta = 0$.
- So the mutation becomes gentle and local.
- **Small η_m:** the distribution spreads out more.

Role of the mutation distribution index η_m

- **Large η_m:** the distribution concentrates near $\delta = 0$.
- So the mutation becomes gentle and local.
- **Small η_m:** the distribution spreads out more.
- So larger perturbations become more likely.

Role of the mutation distribution index η_m

- **Large η_m:** the distribution concentrates near $\delta = 0$.
- So the mutation becomes gentle and local.
- **Small η_m:** the distribution spreads out more.
- So larger perturbations become more likely.
- Therefore η_m controls the aggressiveness of mutation.

Role of the mutation distribution index η_m

- **Large η_m:** the distribution concentrates near $\delta = 0$.
- So the mutation becomes gentle and local.
- **Small η_m:** the distribution spreads out more.
- So larger perturbations become more likely.
- Therefore η_m controls the aggressiveness of mutation.

Note: *SBX controls how offspring are placed between or beyond parents, while polynomial mutation controls how strongly a single solution is perturbed.*

Mutation settings for this example

Assume the following mutation settings:

- number of variables $n = 2$

Mutation settings for this example

Assume the following mutation settings:

- number of variables $n = 2$
- mutation probability $p_m = 1/n = 0.5$

Mutation settings for this example

Assume the following mutation settings:

- number of variables $n = 2$
- mutation probability $p_m = 1/n = 0.5$
- mutation distribution index $\eta_m = 20$

Mutation settings for this example

Assume the following mutation settings:

- number of variables $n = 2$
- mutation probability $p_m = 1/n = 0.5$
- mutation distribution index $\eta_m = 20$

Suppose the mutation decision random numbers for offspring 1 to 8 are:

Mutation settings for this example

Assume the following mutation settings:

- number of variables $n = 2$
- mutation probability $p_m = 1/n = 0.5$
- mutation distribution index $\eta_m = 20$

Suppose the mutation decision random numbers for offspring 1 to 8 are:

$$0.62,\ 0.24,\ 0.98,\ 0.31,\ 0.79,\ 0.71,\ 0.83,\ 0.49$$

Mutation settings for this example

Assume the following mutation settings:

- number of variables $n = 2$
- mutation probability $p_m = 1/n = 0.5$
- mutation distribution index $\eta_m = 20$

Suppose the mutation decision random numbers for offspring 1 to 8 are:

$$0.62,\ 0.24,\ 0.98,\ 0.31,\ 0.79,\ 0.71,\ 0.83,\ 0.49$$

Therefore:

Mutation settings for this example

Assume the following mutation settings:

- number of variables $n = 2$
- mutation probability $p_m = 1/n = 0.5$
- mutation distribution index $\eta_m = 20$

Suppose the mutation decision random numbers for offspring 1 to 8 are:

$$0.62,\ 0.24,\ 0.98,\ 0.31,\ 0.79,\ 0.71,\ 0.83,\ 0.49$$

Therefore:

- offspring 2 mutates
- offspring 4 mutates
- offspring 8 mutates
- the other offspring are copied unchanged

Detailed mutation calculation: offspring 2

Before mutation, offspring 2 is:

$$(-0.801,\ 1.917)^T$$

with bounds $[-5, 5]$ for both variables.

Detailed mutation calculation: offspring 2

Before mutation, offspring 2 is:

$$(-0.801,\ 1.917)^T$$

with bounds $[-5, 5]$ for both variables.
Assume the variable-wise mutation random numbers are:

Detailed mutation calculation: offspring 2

Before mutation, offspring 2 is:

$$(-0.801,\ 1.917)^T$$

with bounds $[-5, 5]$ for both variables.
Assume the variable-wise mutation random numbers are:

$$r_1 = 0.720, \qquad r_2 = 0.0236$$

Detailed mutation calculation: offspring 2

Before mutation, offspring 2 is:

$$(-0.801,\ 1.917)^T$$

with bounds $[-5, 5]$ for both variables.
Assume the variable-wise mutation random numbers are:

$$r_1 = 0.720, \qquad r_2 = 0.0236$$

Then the perturbations are approximately:

Detailed mutation calculation: offspring 2

Before mutation, offspring 2 is:

$$(-0.801,\ 1.917)^T$$

with bounds $[-5, 5]$ for both variables.
Assume the variable-wise mutation random numbers are:

$$r_1 = 0.720, \qquad r_2 = 0.0236$$

Then the perturbations are approximately:

$$\delta_1 \approx 0.0273, \qquad \delta_2 \approx -0.1353$$

Detailed mutation calculation: offspring 2

Before mutation, offspring 2 is:

$$(-0.801,\ 1.917)^T$$

with bounds $[-5, 5]$ for both variables.
Assume the variable-wise mutation random numbers are:

$$r_1 = 0.720, \qquad r_2 = 0.0236$$

Then the perturbations are approximately:

$$\delta_1 \approx 0.0273, \qquad \delta_2 \approx -0.1353$$

Since the variable range is 10, the new values become:

Detailed mutation calculation: offspring 2

Before mutation, offspring 2 is:

$$(-0.801,\ 1.917)^T$$

with bounds $[-5, 5]$ for both variables.
Assume the variable-wise mutation random numbers are:

$$r_1 = 0.720, \qquad r_2 = 0.0236$$

Then the perturbations are approximately:

$$\delta_1 \approx 0.0273, \qquad \delta_2 \approx -0.1353$$

Since the variable range is 10, the new values become:

$$x_1' = -0.801 + 10(0.0273) \approx -0.528$$

Detailed mutation calculation: offspring 2

Before mutation, offspring 2 is:

$$(-0.801,\ 1.917)^T$$

with bounds $[-5, 5]$ for both variables.
Assume the variable-wise mutation random numbers are:

$$r_1 = 0.720, \qquad r_2 = 0.0236$$

Then the perturbations are approximately:

$$\delta_1 \approx 0.0273, \qquad \delta_2 \approx -0.1353$$

Since the variable range is 10, the new values become:

$$x_1' = -0.801 + 10(0.0273) \approx -0.528$$

$$x_2' = 1.917 + 10(-0.1353) \approx 0.564$$

Detailed mutation calculation: offspring 2

Before mutation, offspring 2 is:

$$(-0.801,\ 1.917)^T$$

with bounds $[-5, 5]$ for both variables.
Assume the variable-wise mutation random numbers are:

$$r_1 = 0.720, \qquad r_2 = 0.0236$$

Then the perturbations are approximately:

$$\delta_1 \approx 0.0273, \qquad \delta_2 \approx -0.1353$$

Since the variable range is 10, the new values become:

$$x_1' = -0.801 + 10(0.0273) \approx -0.528$$

$$x_2' = 1.917 + 10(-0.1353) \approx 0.564$$

So the mutated offspring becomes approximately $(-0.528, 0.564)^T$, which is a very strong improvement.

Brief mutation examples for other selected offspring

Offspring 4 before mutation:

$$(2.353,\ 3.010)^T$$

Using suitable random numbers, suppose it mutates to:

$$(1.969,\ 3.140)^T$$

Brief mutation examples for other selected offspring

Offspring 4 before mutation:

$$(2.353,\ 3.010)^T$$

Using suitable random numbers, suppose it mutates to:

$$(1.969,\ 3.140)^T$$

Offspring 8 before mutation:

Brief mutation examples for other selected offspring

Offspring 4 before mutation:

$$(2.353,\ 3.010)^T$$

Using suitable random numbers, suppose it mutates to:

$$(1.969,\ 3.140)^T$$

Offspring 8 before mutation:

$$(-0.809,\ 1.880)^T$$

Brief mutation examples for other selected offspring

Offspring 4 before mutation:

$$(2.353,\ 3.010)^T$$

Using suitable random numbers, suppose it mutates to:

$$(1.969,\ 3.140)^T$$

Offspring 8 before mutation:

$$(-0.809,\ 1.880)^T$$

Suppose it mutates to:

Brief mutation examples for other selected offspring

Offspring 4 before mutation:

$$(2.353,\ 3.010)^T$$

Using suitable random numbers, suppose it mutates to:

$$(1.969,\ 3.140)^T$$

Offspring 8 before mutation:

$$(-0.809,\ 1.880)^T$$

Suppose it mutates to:

$$(0.281,\ 1.850)^T$$

Brief mutation examples for other selected offspring

Offspring 4 before mutation:

$$(2.353,\ 3.010)^T$$

Using suitable random numbers, suppose it mutates to:

$$(1.969,\ 3.140)^T$$

Offspring 8 before mutation:

$$(-0.809,\ 1.880)^T$$

Suppose it mutates to:

$$(0.281,\ 1.850)^T$$

This comparison is useful in class: mutation can improve a solution strongly, improve it slightly, or even make it worse.

Offspring population after mutation

Offspring	x_1	x_2	$f(x_1, x_2)$
1	-2.874	-4.773	17000.594
2	-0.528	0.564	10.470
3	-0.780	1.691	120.371
4	1.969	3.140	55.250
5	-0.639	1.692	167.469
6	2.212	3.009	356.393
7	-2.222	-2.342	5309.179
8	0.281	1.850	314.175

Summary after mutation

- offspring 2 improves dramatically after mutation

Summary after mutation

- offspring 2 improves dramatically after mutation
- offspring 4 also improves

Summary after mutation

- offspring 2 improves dramatically after mutation
- offspring 4 also improves
- offspring 8 becomes worse than before mutation

Summary after mutation

- offspring 2 improves dramatically after mutation
- offspring 4 also improves
- offspring 8 becomes worse than before mutation

Note: mutation is not guaranteed to improve a solution. Its purpose is to inject perturbation and diversity. Selection later decides which solutions deserve to survive.

Survivor selection: $(\mu + \lambda)$ strategy

To form the next generation, we combine:

- the parent population of size $\mu = 8$

Survivor selection: $(\mu + \lambda)$ strategy

To form the next generation, we combine:

- the parent population of size $\mu = 8$
- the offspring population of size $\lambda = 8$

Survivor selection: $(\mu + \lambda)$ strategy

To form the next generation, we combine:

- the parent population of size $\mu = 8$
- the offspring population of size $\lambda = 8$

Then we sort all 16 solutions by objective value and keep the best 8.

Survivor selection: $(\mu + \lambda)$ strategy

To form the next generation, we combine:

- the parent population of size $\mu = 8$
- the offspring population of size $\lambda = 8$

Then we sort all 16 solutions by objective value and keep the best 8.

This is called the $(\mu + \lambda)$ strategy. Strong parents can survive, and strong offspring can replace weak parents.

Survivor selection: ranking the combined population

The best solutions after combining parents and offspring are:

Rank	Source	Index	(x_1, x_2)	Objective
1	Offspring	2	$(-0.528, 0.564)$	10.470
2	Offspring	4	$(1.969, 3.140)$	55.250
3	Offspring	3	$(-0.780, 1.691)$	120.371
4	Parent	4	$(-0.639, 1.692)$	167.469
5	Offspring	5	$(-0.639, 1.692)$	167.469
6	Parent	7	$(-0.742, 1.934)$	194.424
7	Offspring	8	$(0.281, 1.850)$	314.175
8	Parent	1	$(2.212, 3.009)$	356.393

Next-generation parent population

So the next parent population becomes:

New index	x_1	x_2	$f(x_1, x_2)$
1	-0.528	0.564	10.470
2	1.969	3.140	55.250
3	-0.780	1.691	120.371
4	-0.639	1.692	167.469
5	-0.639	1.692	167.469
6	-0.742	1.934	194.424
7	0.281	1.850	314.175
8	2.212	3.009	356.393

Next-generation parent population

So the next parent population becomes:

New index	x_1	x_2	$f(x_1, x_2)$
1	-0.528	0.564	10.470
2	1.969	3.140	55.250
3	-0.780	1.691	120.371
4	-0.639	1.692	167.469
5	-0.639	1.692	167.469
6	-0.742	1.934	194.424
7	0.281	1.850	314.175
8	2.212	3.009	356.393

This becomes the parent population for generation $t = 2$.

What changed and what stayed the same from BGA to RGA?

Aspect	BGA	RGA
Representation	binary strings	real-valued vectors
Selection	based on fitness	based on fitness
Crossover	bit-level crossover	SBX or other real-valued crossover
Mutation	bit flip	polynomial or other real perturbation
Precision	depends on bit length	direct real values
Decoding	required	not required

What changed and what stayed the same from BGA to RGA?

Aspect	BGA	RGA
Representation	binary strings	real-valued vectors
Selection	based on fitness	based on fitness
Crossover	bit-level crossover	SBX or other real-valued crossover
Mutation	bit flip	polynomial or other real perturbation
Precision	depends on bit length	direct real values
Decoding	required	not required

This comparison is worth emphasizing: selection logic remains the same, but representation and variation operators are redesigned.

Discussion questions

- Why is binary coding inefficient for many continuous optimization problems?

Discussion questions

- Why is binary coding inefficient for many continuous optimization problems?
- Why does selection remain almost unchanged in RGA?

Discussion questions

- Why is binary coding inefficient for many continuous optimization problems?
- Why does selection remain almost unchanged in RGA?
- What does the spread factor β mean in SBX?

Discussion questions

- Why is binary coding inefficient for many continuous optimization problems?
- Why does selection remain almost unchanged in RGA?
- What does the spread factor β mean in SBX?
- How does η_c affect exploration and exploitation?

Discussion questions

- Why is binary coding inefficient for many continuous optimization problems?
- Why does selection remain almost unchanged in RGA?
- What does the spread factor β mean in SBX?
- How does η_c affect exploration and exploitation?
- Why is mutation probability usually kept small in genetic algorithms?

Discussion questions

- Why is binary coding inefficient for many continuous optimization problems?
- Why does selection remain almost unchanged in RGA?
- What does the spread factor β mean in SBX?
- How does η_c affect exploration and exploitation?
- Why is mutation probability usually kept small in genetic algorithms?
- Why can mutation both improve and worsen a solution?

Quick quiz

1. In RGA, chromosomes are usually represented as:

- A. fixed binary strings only

Quick quiz

1. In RGA, chromosomes are usually represented as:

- A. fixed binary strings only
- B. real-valued vectors

Quick quiz

1. In RGA, chromosomes are usually represented as:

- A. fixed binary strings only
- B. real-valued vectors
- C. permutations only

Quick quiz

1. In RGA, chromosomes are usually represented as:

- A. fixed binary strings only
- B. real-valued vectors
- C. permutations only
- D. logic gates

Quick quiz

1. In RGA, chromosomes are usually represented as:

- A. fixed binary strings only
- B. real-valued vectors
- C. permutations only
- D. logic gates

Answer: B

Quick quiz

1. In RGA, chromosomes are usually represented as:

- A. fixed binary strings only
- B. real-valued vectors
- C. permutations only
- D. logic gates

Answer: B

2. Which operator is commonly used as real-valued crossover in RGA?

- A. bit-flip mutation
- B. roulette wheel decoding

Quick quiz

1. In RGA, chromosomes are usually represented as:

- A. fixed binary strings only
- B. real-valued vectors
- C. permutations only
- D. logic gates

Answer: B

2. Which operator is commonly used as real-valued crossover in RGA?

- A. bit-flip mutation
- B. roulette wheel decoding
- C. simulated binary crossover

Quick quiz

1. In RGA, chromosomes are usually represented as:

- A. fixed binary strings only
- B. real-valued vectors
- C. permutations only
- D. logic gates

Answer: B

2. Which operator is commonly used as real-valued crossover in RGA?

- A. bit-flip mutation
- B. roulette wheel decoding
- C. simulated binary crossover
- D. single-point bit swap

Quick quiz

1. In RGA, chromosomes are usually represented as:

- A. fixed binary strings only
- B. real-valued vectors
- C. permutations only
- D. logic gates

Answer: B

2. Which operator is commonly used as real-valued crossover in RGA?

- A. bit-flip mutation
- B. roulette wheel decoding
- C. simulated binary crossover
- D. single-point bit swap

Quick quiz

1. In RGA, chromosomes are usually represented as:

- A. fixed binary strings only
- B. real-valued vectors
- C. permutations only
- D. logic gates

Answer: B

2. Which operator is commonly used as real-valued crossover in RGA?

- A. bit-flip mutation
- B. roulette wheel decoding
- C. simulated binary crossover
- D. single-point bit swap

Quick quiz

1. In RGA, chromosomes are usually represented as:

- A. fixed binary strings only
- B. real-valued vectors
- C. permutations only
- D. logic gates

Answer: B

2. Which operator is commonly used as real-valued crossover in RGA?

- A. bit-flip mutation
- B. roulette wheel decoding
- C. simulated binary crossover
- D. single-point bit swap

Answer: C

Quick quiz

3. In polynomial mutation, the parameter η_m mainly controls:

- A. population size

Quick quiz

3. In polynomial mutation, the parameter η_m mainly controls:

- A. population size
- B. mutation perturbation distribution

Quick quiz

3. In polynomial mutation, the parameter η_m mainly controls:

- A. population size
- B. mutation perturbation distribution
- C. tournament size

Quick quiz

3. In polynomial mutation, the parameter η_m mainly controls:

- A. population size
- B. mutation perturbation distribution
- C. tournament size
- D. number of parents only

Quick quiz

3. In polynomial mutation, the parameter η_m mainly controls:

- A. population size
- B. mutation perturbation distribution
- C. tournament size
- D. number of parents only

Answer: B

Quick quiz

3. In polynomial mutation, the parameter η_m mainly controls:

- A. population size
- B. mutation perturbation distribution
- C. tournament size
- D. number of parents only

Answer: B

4. In the $(\mu + \lambda)$ strategy, the next generation is selected from:

- A. parents only
- B. offspring only

Quick quiz

3. In polynomial mutation, the parameter η_m mainly controls:

- A. population size
- B. mutation perturbation distribution
- C. tournament size
- D. number of parents only

Answer: B

4. In the $(\mu + \lambda)$ strategy, the next generation is selected from:

- A. parents only
- B. offspring only
- C. both parents and offspring together

Quick quiz

3. In polynomial mutation, the parameter η_m mainly controls:

- A. population size
- B. mutation perturbation distribution
- C. tournament size
- D. number of parents only

Answer: B

4. In the $(\mu + \lambda)$ strategy, the next generation is selected from:

- A. parents only
- B. offspring only
- C. both parents and offspring together
- D. randomly generated points only

Quick quiz

3. In polynomial mutation, the parameter η_m mainly controls:

- A. population size
- B. mutation perturbation distribution
- C. tournament size
- D. number of parents only

Answer: B

4. In the $(\mu + \lambda)$ strategy, the next generation is selected from:

- A. parents only
- B. offspring only
- C. both parents and offspring together
- D. randomly generated points only

Quick quiz

3. In polynomial mutation, the parameter η_m mainly controls:

- A. population size
- B. mutation perturbation distribution
- C. tournament size
- D. number of parents only

Answer: B

4. In the $(\mu + \lambda)$ strategy, the next generation is selected from:

- A. parents only
- B. offspring only
- C. both parents and offspring together
- D. randomly generated points only

Quick quiz

3. In polynomial mutation, the parameter η_m mainly controls:

- A. population size
- B. mutation perturbation distribution
- C. tournament size
- D. number of parents only

Answer: B

4. In the $(\mu + \lambda)$ strategy, the next generation is selected from:

- A. parents only
- B. offspring only
- C. both parents and offspring together
- D. randomly generated points only

Answer: C

End-of-lecture summary

Key takeaways from this lecture:

- binary coding may be inconvenient for real-parameter optimization

End-of-lecture summary

Key takeaways from this lecture:

- binary coding may be inconvenient for real-parameter optimization
- RGA uses real-valued chromosomes directly

End-of-lecture summary

Key takeaways from this lecture:

- binary coding may be inconvenient for real-parameter optimization
- RGA uses real-valued chromosomes directly
- binary tournament selection works in the same way as before

End-of-lecture summary

Key takeaways from this lecture:

- binary coding may be inconvenient for real-parameter optimization
- RGA uses real-valued chromosomes directly
- binary tournament selection works in the same way as before
- SBX performs crossover directly on real numbers

End-of-lecture summary

Key takeaways from this lecture:

- binary coding may be inconvenient for real-parameter optimization
- RGA uses real-valued chromosomes directly
- binary tournament selection works in the same way as before
- SBX performs crossover directly on real numbers
- polynomial mutation perturbs real values directly

End-of-lecture summary

Key takeaways from this lecture:

- binary coding may be inconvenient for real-parameter optimization
- RGA uses real-valued chromosomes directly
- binary tournament selection works in the same way as before
- SBX performs crossover directly on real numbers
- polynomial mutation perturbs real values directly
- one full generation consists of evaluation, selection, crossover, mutation, evaluation, and survivor selection

End-of-lecture summary

Key takeaways from this lecture:

- binary coding may be inconvenient for real-parameter optimization
- RGA uses real-valued chromosomes directly
- binary tournament selection works in the same way as before
- SBX performs crossover directly on real numbers
- polynomial mutation perturbs real values directly
- one full generation consists of evaluation, selection, crossover, mutation, evaluation, and survivor selection
- the $(\mu + \lambda)$ strategy chooses the best solutions from parents and offspring together